

PDF-Bild-OCR: Installationsanleitung für den Firmenrechner

Oct 1, 2026 · @Holger

Zweck und Überblick

`pdf_image_ocr.py` (Version 1.1.0) macht englischen Text in eingebetteten PDF-Bildern durchsuchbar, komplett offline auf dem eigenen Rechner. Der vorhandene PDF-Text bleibt unverändert; das Skript legt nur über die Bildbereiche eine unsichtbare Textebene.

Für die Installation per pip werden nur zwei Pakete direkt benötigt:

Paket	Aufgabe	Bringt mit
<code>pymupdf</code>	PDF lesen, Bildbereiche finden, unsichtbaren Text schreiben	-
<code>rapidocr_onnxruntime</code>	Texterkennung (OCR) inkl. Modelle	<code>onnxruntime</code> , <code>numpy</code> , <code>opencv-python</code> , weitere Hilfspakete

PyTorch und EasyOCR werden **nicht** benötigt. Sie werden von der Anwendungssteuerung des Firmenrechners blockiert (WinError 4551) und waren der Grund für den Wechsel auf RapidOCR.

Voraussetzungen

Getestet wurde auf Windows 64-Bit mit Python 3.14 (64-Bit), PowerShell und onnxruntime 1.30.0.

- **Python 64-Bit** ist installiert. Prüfen mit `python --version` und `python -c "import struct; print(struct.calcsize('P')*8)"` (Ausgabe muss 64 sein).
- **pip** ist verfügbar: `python -m pip --version`.
- **Paketquelle**: entweder Internetzugang für pip, ein internes Paket-Repository (z. B. Artifactory/Nexus) oder ein Ordner mit vorab heruntergeladenen Paketen (siehe „Variante ohne Internet“).
- **Anwendungssteuerung**: Die DLLs von `onnxruntime` und `opencv` dürfen geladen werden. Auf dem Firmenrechner ist das geprüft und funktioniert.

Installation per pip

Die Installation besteht aus einem pip-Befehl und einem Prüfbefehl. Alle Befehle in PowerShell ausführen.

1. pip aktualisieren:

```
python -m pip install --upgrade pip
```

2. Pakete installieren: Die Pakete werden direkt in die vorhandene Python-Installation installiert, ohne virtuelle Umgebung und ohne `Activate.ps1`.

```
python -m pip install pymupdf rapidocr_onnxruntime
```

3. Installation prüfen:

```
python -c "import pymupdf, onnxruntime; from rapidocr_onnxruntime import
RapidOCR; print(pymupdf.VersionBind, onnxruntime.__version__,
onnxruntime.get_device())"
```

Erwartet wird eine Zeile wie `1.28.2 1.30.0 CPU`. Eine Fehlermeldung hier deutet auf ein Problem hin, das unter „Fehlerbehebung“ beschrieben ist.

Für die Dokumentation der eingesetzten Versionen kann man den Stand festhalten:

`python -m pip freeze > requirements_ocr.txt`. Mit dieser Datei lässt sich dieselbe Umgebung später per `python -m pip install -r requirements_ocr.txt` wiederherstellen.

Variante ohne Internet

Hat der Zielrechner keinen Internetzugang, lädt man die Pakete auf einem anderen Rechner herunter und installiert sie aus einem Ordner. Der Download-Rechner muss **dieselbe Python-Version** (z. B. 3.14) und ebenfalls **Windows 64-Bit** haben, sonst passen die Pakete nicht.

1. Auf dem Rechner mit Internet:

```
python -m pip download pymupdf rapidocr_onnxruntime -d wheels
```

2. Den Ordner `wheels` auf den Firmenrechner übertragen (gemäß eurem Freigabeprozess).

3. Auf dem Firmenrechner, ohne Zugriff auf das Internet:

```
python -m pip install --no-index --find-links wheels pymupdf
rapidocr_onnxruntime
```

4. Installation wie oben mit dem Prüfbefehl testen.

Die OCR-Modelle sind im Paket `rapidocr_onnxruntime` enthalten. Ein separater Modell-Download ist nicht nötig. Für die Freigabe lassen sich Prüfsummen der Pakete festhalten:

```
Get-FileHash wheels\* | Format-Table Hash, Path.
```

Skript ablegen und aufrufen

Das Skript ist eine einzelne Datei, `pdf_image_ocr.py`. Sie kann in einem beliebigen Ordner liegen; ein Modellordner wird nicht mehr benötigt.

Einzelne Datei verarbeiten:

```
python pdf_image_ocr.py dokument.pdf
```

Es entsteht `dokument_ocr.pdf` im selben Ordner. Das Original wird nie verändert. Pfade mit Leerzeichen in Anführungszeichen setzen.

Ganzen Ordner inklusive Unterordnern verarbeiten, mit Bericht und Logdatei:

```
python pdf_image_ocr.py C:\Docs -r -o C:\Docs_OCR --report bericht.json --log ocr.log
```

Option	Wirkung	Standard
<code>-o, --out-dir</code>	Zielordner für die Ergebnisse	neben der Quelldatei
<code>--suffix</code>	Anhang an den Dateinamen	<code>_ocr</code>
<code>-r, --recursive</code>	Unterordner mit durchsuchen	aus
<code>--overwrite</code>	Vorhandene Ergebnisse überschreiben	aus (wird übersprungen)
<code>--dpi</code>	Auflösung für die OCR; höher hilft bei kleiner Schrift	300
<code>--min-conf</code>	Mindest-Erkennungssicherheit (0-1); niedriger findet mehr, aber auch Fehltreffer	0.5
<code>--min-image-size</code>	Kleinere Bilder (in pt) ignorieren, z. B. Icons	20
<code>--no-verify</code>	Verifikation abschalten (nicht empfohlen)	Verifikation an
<code>--report</code>	JSON-Bericht schreiben	-

Option	Wirkung	Standard
<code>--log</code>	Logdatei schreiben	–
<code>-v</code> , <code>--verbose</code>	Ausführliche Ausgabe pro Seite	aus
<code>--version</code>	Skriptversion anzeigen	–

Ergebnis prüfen

Jede Datei wird nach der Verarbeitung automatisch geprüft; in der Konsole steht danach `OK` oder `PROBLEM`.

- **Wortprüfung:** Alle Original-Wörter müssen an derselben Position im Ergebnis vorhanden sein.
- **Optische Prüfung:** Jede Seite muss pixelgleich zum Original aussehen, denn der unsichtbare Text darf das Erscheinungsbild nicht verändern.
- **Exit-Code:** `0` = alles in Ordnung, `1` = mindestens eine Datei mit Problem, `2` = Start nicht möglich (z. B. keine PDFs gefunden, OCR-Engine fehlt). In PowerShell abfragbar mit `$LASTEXITCODE`.

Zusätzlich empfiehlt sich ein manueller Suchtest: Im Ergebnis-PDF nach einem Wort suchen, das nur in einem Bild vorkommt, und nach einem zweiten Wort direkt daneben. Werden beide einzeln gefunden, sind auch die Leerzeichen korrekt erkannt. Das Standardmodell von RapidOCR kann bei englischem Text gelegentlich Leerzeichen auslassen („SafetyRequirement“ statt „Safety Requirement“).

Fehlerbehebungsmach